Kurtosys Studio Product-Page Monorepo — User Guide

Repository: `ksys-studio-pp-template` · Companion widget starter: `ksys-studio-widget-template`

One monorepo for product-page widgets. The first decision is **where they embed** (Kurtosys DXM vs external CMS). That choice factors builds, deploy paths, and docs — without forking the widget source.

Contents

1. Step 0 — Choose host mode (start here)
2. What this repo is
3. Prerequisites
4. Repository layout
5. Initialize a client project
6. DXM vs CMS — how the project is factored
7. Manage apps (list / add / rename)
8. Develop a single app
9. Configure environments
10. DXM mount contract
11. CMS integration specs
12. Build for dev / staging / prod
13. Built-in defaults
14. Security audit reports
15. End-to-end checklist
16. Troubleshooting
17. Regenerate this PDF

Step 0 — Choose host mode (start here)

> **Do this first.** Before adding features or filling GUIDs, decide how the widgets will be embedded. Run init and answer the host question (or pass flags).

```
npm run init
# interactive prompts: host mode → client slug → CMS platform (if needed)

# or non-interactive:
npm run init -- --host dxm --client acme
npm run init -- --host cms --client acme --cms-platform pantheon
npm run init -- --host both --client acme --cms-platform wordpress
```

hostMode	Use when	Bundle behaviour	Primary docs / output
`dxm`	Kurtosys DXM / Studio / Elementor pages	IIFE, **React external**	`hosts/dxm/` · `BuildResult/dxm/…`
`cms`	Client WordPress / Pantheon / custom CMS	IIFE, **React bundled** (self-contained)	`hosts/cms/` · integration specs · `BuildResult/cms/…`
`both`	Same widgets delivered to DXM *and* CMS	Builds both variants from one source	Both host folders · dual `BuildResult` trees

> You do **not** need two monorepos. Host mode only changes packaging, deploy path, and documentation. Widget UI/source stays under `development_v1/product-page-apps/`.

Re-run later with `npm run init -- --host cms --force` (or `npm run host:set`) if the delivery model changes.

1. What this repo is

A **generic, reusable monorepo** for embeddable React product-page widgets. Each widget is a Vite app under `development_v1/product-page-apps/`, registered in `studio.config.json`.

Starter apps:

App id	Mount id	Embed global	Dev port
`pageHeader`	`#pageHeader`	`window.pageHeader`	5173
`overview`	`#overview`	`window.overview`	5174
`performance`	`#performance`	`window.performanceApp` *	5175
`portfolioBreakdown`	`#portfolioBreakdown`	`window.portfolioBreakdown`	5176
`literature`	`#literature`	`window.literature`	5177

> * `window.performance` is reserved by the browser; the loader uses `performanceApp` automatically.

2. Prerequisites

- **Node.js** 20+ and **npm**
- **Python 3.10+** for security reports and PDF generation
- Sibling `ksys-studio-widget-template` (for `npm run apps:add`)
- Optional: Chrome / Edge for PDF export

3. Repository layout

```
ksys-studio-pp-template/
├── USER_GUIDE.html / USER_GUIDE.pdf
├── studio.config.json          ← hostMode + apps registry
├── hosts/
│   ├── ACTIVE_HOST_MODE.txt
│   ├── dxm/                    ← DXM checklist & notes
│   └── cms/                    ← CMS embed notes
├── docs/
│   ├── ACCESSIBILITY.md
│   └── integration-specs/      ← one markdown spec per CMS product page
├── scripts/                    ← init, add/rename, build-all
├── tools/                      ← security audit + user-guide PDF
├── BuildResult/
│   ├── dxm/{dev,staging,prod}/
│   └── cms/{dev,staging,prod}/
└── development_v1/product-page-apps/
    └── ksys-<client>-<appId>/
```

4. Initialize a client project

1. Copy the template next to the widget starter:

```
cp -R ~/Documents/GitHub/ksys-studio-pp-template ~/Documents/GitHub/acme-pp
cd ~/Documents/GitHub/acme-pp
```

2. **Init host mode** (Step 0).
3. If the client slug is not still `client`, rename folders:

```
npm run client:set -- acme
```

   (If you already passed `--client acme` to init, still run `client:set` when folders were created as `ksys-client-*`.)
4. `npm run apps:list` — confirm `hostMode` and app folders.
5. Fill each app's `config/env.config.cjs`.

5. DXM vs CMS — how the project is factored

Shared (always)

- One source tree per widget
- A11y defaults, Tailwind `important`, security pins
- App add / rename / client slug tooling

DXM-only factoring

- `HOST_MODE=dxm` → React **externalized**
- Studio GUIDs + Fundpress / applicationManager URLs
- Outputs under `BuildResult/dxm/<env>/`
- Guidance in `hosts/dxm/README.md`

CMS-only factoring

- `HOST_MODE=cms` → React **bundled** (self-contained for WP/Pantheon)
- Integration spec per product page (Confluence-style)
- Outputs under `BuildResult/cms/<env>/`
- Guidance in `hosts/cms/README.md`

`both`

`npm run build:staging` builds each app twice and collects into both `BuildResult/dxm/…` and `BuildResult/cms/…`.

6. Manage apps

```
npm run apps:list
npm run apps:add -- holdingsExposure 5178
npm run apps:rename -- literature materials
npm run client:set -- newclient
```

Folder convention: `ksys-<clientSlug>-<appId>/`

7. Develop a single app

```
npm run dev -- overview
```

Local `index.html` simulates App Manager + mount node for DXM-style inputs. For CMS work, the same mount `data-input-*` attributes are still the preferred contract.

8. Configure environments

Each app `config/env.config.cjs`: `development` / `staging` / `production`.

- `guid`, `apiToken`, `savedSearchToken`
- `filename`, `containerId`, `embedGlobalName`

> Do not commit production secrets to public remotes.

9. DXM mount contract

```
<div
  data-ksys-app-template-id="ksys-app-manager"
  data-input-client-code="…"
  data-input-culture="en-US"
  data-input-product-type="mutualFunds"
></div>
<div id="overview"></div>
<script src="…/overview.js"></script>
```

See `hosts/dxm/README.md`.

10. CMS integration specs

For WordPress / Pantheon (and similar), create **one integration spec per product page** — the same idea as your Confluence “Product Page Strategies / All Regions” templates.

1. Copy `docs/integration-specs/_TEMPLATE.md`
2. Rename to the product page (and region if needed)
3. Fill mount selectors, script URLs, cultures, QA checklists

```
cp docs/integration-specs/_TEMPLATE.md \
   docs/integration-specs/acme-global-equity-us.md
```

See `hosts/cms/README.md` for embed snippets.

11. Build for dev / staging / prod

```
# uses studio.config.json hostMode
npm run build:dev
npm run build:staging
npm run build:prod

# force a single host even if mode is "both"
npm run build:staging:dxm
npm run build:staging:cms
```

Artifacts:

```
BuildResult/dxm/staging/overview/overview.js
BuildResult/cms/staging/overview/overview.js
```

> Per-app scripts also exist: `build:dxm:staging`, `build:cms:staging`, etc. Monorepo `build-all` sets `HOST_MODE` and prefers those scripts.

12. Built-in defaults

- **A11y** — WCAG-oriented landmarks, contrast tokens, focus rings, jsx-a11y (see `docs/ACCESSIBILITY.md`)
- **Tailwind** — `@import "tailwindcss" important;`
- **Embed global** — `window.<appId>` (not `YourAppEmbed`)
- **Security pins** — vite, rollup, tar, picomatch, postcss

13. Security audit reports

```
pip install -r tools/requirements-audit-report.txt
npm run audit:security
```

HTML/PDF under `reports/`, scoped to app packages and `BuildResult` bundles.

14. End-to-end checklist

1. `npm run init -- --host dxm|cms|both --client <slug>`
2. `npm run client:set -- <slug>` if folders still use the old slug
3. Add/rename apps as needed
4. Fill `env.config.cjs`
5. If CMS/both: create integration specs from `_TEMPLATE.md`
6. Implement UI; keep a11y patterns
7. `npm run build:staging` → verify `BuildResult/<host>/…`
8. Deploy (Studio upload and/or CMS CDN)
9. `npm run audit:security`

15. Troubleshooting

Symptom	Fix
`hostMode is not set` on build	Run `npm run init` first
CMS page has no React / blank widget	Build with `HOST_MODE=cms` (React must be bundled)
DXM double-React / hook errors	Use `HOST_MODE=dxm` (React externalized)
Styles overridden by theme / Elementor	Confirm Tailwind `important` import in `src/index.css`
`apps:add` cannot find template	Fix `widgetTemplatePath` in `studio.config.json`

16. Regenerate this PDF

```
npm run docs:user-guide
# or: python3 tools/build_user_guide_pdf.py
```

Edit `USER_GUIDE.html` (source of truth), then regenerate the PDF.

Kurtosys Studio product-page monorepo · User guide · Start with `npm run init` so host mode factors the rest of the project correctly.